INFO 5 – Projet 2 : Cryptographie élémentaire

Niveau : L1 Maths | Séance 4 sur 5 (3h)

Chiffrement de Vigenère et attaque par force brute

Objectifs de la séance

- Comprendre le principe du chiffrement de Vigenère.
- Utiliser un mot-clé pour chiffrer un message.
- Manipuler les caractères en Python avec `ord` et `chr`.
- Utiliser l'opérateur modulo pour répéter automatiquement une clé.
- Implémenter le chiffrement et le déchiffrement de Vigenère.
- Découvrir, en bonus, une attaque par force brute sur une clé courte.

Partie 1 : Le principe de Vigenère

Le défaut du chiffrement de César et du chiffrement affine est assez simple : une même lettre en clair donne toujours la même lettre chiffrée.

Par exemple, avec César, si la lettre `E` est chiffrée en `H`, alors tous les `E` du message deviennent des `H`. Cela laisse beaucoup d'informations visibles dans le texte chiffré, surtout si le message est long.

Le chiffrement de Vigenère, attribué à Blaise de Vigenère au XVIe siècle, améliore cette idée en utilisant un **mot-clé**. Au lieu d'appliquer toujours le même décalage, on applique un décalage qui dépend des lettres de la clé.

Pour chiffrer la i-ème lettre du message, notée M_i, on utilise la i-ème lettre de la clé, notée K_i :

$$C_i = (M_i + K_i) \pmod{26}$$

Si le mot-clé est plus court que le message, on le répète en boucle.

Par exemple, avec la clé `MATHS` :

Message	B	O	N	J	O	U	R	A	T	O
Clé	M	A	T	H	S	M	A	T	H	S

La sixième lettre du message est donc chiffrée avec la première lettre de la clé, car la clé recommence.

0.0.1 Un exemple guidé

On considère le message :

`MATH`

et la clé :

`CLE`

La clé est répétée ainsi :

Message	M	A	T	H
Clé	C	L	E	C

Les positions sont :

Message	M	A	T	H
Position message	12	0	19	7
Clé	C	L	E	C
Position clé	2	11	4	2

On chiffre chaque lettre :

$$\begin{aligned} M &: (12+2) \mod 26 = 14 \Rightarrow O \\ A &: (0+11) \mod 26 = 11 \Rightarrow L \\ T &: (19+4) \mod 26 = 23 \Rightarrow X \\ H &: (7+2) \mod 26 = 9 \Rightarrow J \end{aligned}$$

Donc :

`MATH` devient `OLXJ`

Exercice 1 : Les outils de base

Pour simplifier, nous allons supposer que les lettres sont en majuscules.
Écrivez deux fonctions :

- `lettre_vers_pos(lettre)` : transforme une lettre en nombre, avec `A = 0`, `B = 1`, etc.
- `pos_vers_lettre(pos)` : transforme un nombre entre 0 et 25 en lettre.

```python
def lettre_vers_pos(lettre):
    return ord(lettre) - ord('A')

def pos_vers_lettre(pos):
    return chr(pos + ord('A'))
```

Testez vos fonctions :

```python
print(lettre_vers_pos('A'))  # doit afficher 0
print(lettre_vers_pos('C'))  # doit afficher 2
print(pos_vers_lettre(0))    # doit afficher A
print(pos_vers_lettre(25))   # doit afficher Z
```

Partie 2 : Chiffrement avec un mot-clé

Dans cette partie, on veut écrire une fonction qui chiffre un texte avec une clé.
La difficulté principale est de choisir la bonne lettre de la clé à chaque étape. Si la clé est plus courte que le texte, on doit revenir au début de la clé.
Pour cela, on utilise l'opérateur modulo.
Si la clé est `MATHS`, sa longueur est 5. Les indices utilisés seront :

$$0, 1, 2, 3, 4, 0, 1, 2, 3, 4, \ldots$$

En Python, cela s'écrit :

```python
indice_cle = index_lettre % len(cle)
```

Remarque importante

On ne chiffre que les lettres. Les espaces, la ponctuation et les autres caractères sont gardés tels quels.

Par exemple :

`BON JOUR` peut rester avec son espace.

Mais attention : les espaces ne doivent pas faire avancer la clé. C'est pour cela qu'on utilise un compteur `index_lettre`, qui avance seulement quand on rencontre une lettre.

Exercice 2 : Chiffrer avec Vigenère

Complétez la fonction suivante :

```
def chiffrer_vigenere(texte, cle):
    """Chiffre un texte avec le chiffrement de Vigenere."""


    return resultat
```

Testez votre fonction :

```
message = "BONJOUR"
cle = "MATHS"

texte_chiffre = chiffrer_vigenere(message, cle)

print(texte_chiffre)
```

Question

Que se passe-t-il si vous chiffrez le message :

`BON JOUR`

avec un espace au milieu ? Le résultat garde-t-il l'espace ?

Partie 3 : Déchiffrement de Vigenère

Pour déchiffrer, on fait l'opération inverse.
La formule de chiffrement était :

$$C_i = (M_i + K_i) \pmod{26}$$

Donc la formule de déchiffrement est :

$$M_i = (C_i - K_i) \pmod{26}$$

En Python, l'opérateur `%` fonctionne aussi avec les nombres négatifs. Par exemple :

```
print((-3) % 26)
```

Le résultat est `23`, ce qui correspond bien à une position valide dans l'alphabet.

Exercice 3 : Déchiffrer avec Vigenère

Écrivez la fonction `dechiffrer_vigenere(texte_chiffre, cle)`.

Vous pouvez reprendre presque la même structure que pour le chiffrement. La seule différence est la formule :

$$m = (c - k) \pmod{26}$$

Code à compléter :

```
def dechiffrer_vigenere(texte_chiffre, cle):
    """Dechiffre un texte chiffre avec Vigenere."""
    texte_chiffre = texte_chiffre.upper()
    cle = cle.upper()

    resultat = ""
    index_lettre = 0

    return resultat
```

Test complet

Testez votre programme avec le code suivant :

```
message = "BONJOUR A TOUS"
cle = "MATHS"

texte_chiffre = chiffrer_vigenere(message, cle)
texte_dechiffre = dechiffrer_vigenere(texte_chiffre, cle)

print("Message initial  :", message)
print("Texte chiffre    :", texte_chiffre)
print("Texte dechiffre  :", texte_dechiffre)
```

Normalement, le texte déchiffré doit être identique au message initial, à part éventuellement les majuscules.

Partie 4 : – Attaque de Vigenère par force brute

L'analyse fréquentielle globale fonctionne moins bien avec Vigenère, car une même lettre du message peut être chiffrée de plusieurs façons différentes.

Cependant, si on sait que la clé est très courte, on peut essayer toutes les clés possibles.

Par exemple, une clé de 3 lettres donne :

$$26 \times 26 \times 26 = 17576$$

possibilités. Pour un ordinateur, cela reste raisonnable.

Exercice 4 : Générer toutes les clés de longueur 3

Complétez la fonction suivante. Elle doit générer toutes les clés de 3 lettres, de `AAA` à `ZZZ`.

```
def generer_cles_3():
    cles = []


    return cles
```

Testez :

```
cles = generer_cles_3()

print(len(cles))      # doit afficher 17576
print(cles[0])        # AAA
print(cles[-1])       # ZZZ
```

Exercice 5 : Tester toutes les clés

On suppose qu'un texte a été chiffré avec une clé de longueur 3.
L'idée est simple :

— générer toutes les clés possibles ;

— déchiffrer le texte avec chaque clé ;

— afficher les résultats les plus prometteurs.

Pour commencer, on peut afficher tous les résultats, même si ce n'est pas très pratique :

```
def brute_force_vigenere_3(texte_chiffre):
    cles = generer_cles_3()

    for cle in cles:
        texte_possible = dechiffrer_vigenere(texte_chiffre, cle)
        print(cle, ":", texte_possible)
```

Test :

```
message = "CE␣MESSAGE␣EST␣SECRET"
cle = "BAC"

texte_chiffre = chiffrer_vigenere(message, cle)

print("Texte␣chiffre␣:", texte_chiffre)

brute_force_vigenere_3(texte_chiffre)
```

Amélioration possible

Afficher 17576 lignes n'est pas très lisible. On peut donc utiliser un score simple.

Par exemple, si un texte contient souvent des mots comme `LE`, `LA`, `DE`, `ET`, `EST`, il a plus de chances d'être un texte français.

Voici une fonction très simple :

```
def score_francais(texte):
    mots_frequents = ["LE", "LA", "LES", "DE", "DES", "ET", "EST", "UN", "UNE", "QUE"
        ]

    return score
```

On peut alors garder seulement les meilleurs résultats :

```
def brute_force_vigenere_3_score(texte_chiffre):
```

Test final :

```
message = "CE␣MESSAGE␣EST␣SECRET"
cle = "BAC"

texte_chiffre = chiffrer_vigenere(message, cle)

print("Texte␣chiffre␣:", texte_chiffre)
print("Meilleures␣propositions␣:")

brute_force_vigenere_3_score(texte_chiffre)
```

Partie 5 : Bonus – Majuscules, minuscules et ASCII

Jusqu'ici, nous avons travaillé uniquement avec les majuscules. Le décalage se faisait donc modulo 26.

On peut maintenant gérer à la fois les minuscules et les majuscules. Pour cela, on va utiliser les fonctions `ord` et `chr`, mais sans écrire directement un alphabet sous forme de chaîne.

On choisit l'ordre suivant :

`a, b, c, ..., z, A, B, C, ..., Z`

Il y a donc 52 caractères. Le décalage se fera modulo 52.

Par exemple, avec cet ordre :

`z + 1 = A`

et :

`z + 2 = B`

Exercice 6 : Position d'une lettre

Complétez les fonctions suivantes :

```
def caractere_vers_pos(caractere):

    return ....
```

```
def pos_vers_caractere(pos):

    return...
```

Testez :

```
print(caractere_vers_pos('a'))  # 0
print(caractere_vers_pos('z'))  # 25
print(caractere_vers_pos('A'))  # 26
print(caractere_vers_pos('B'))  # 27

print(pos_vers_caractere(0))    # a
print(pos_vers_caractere(25))   # z
print(pos_vers_caractere(26))   # A
print(pos_vers_caractere(27))   # B
```

Exercice 7 : Chiffrement avec minuscules et majuscules

On écrit maintenant une version du chiffrement de Vigenère qui garde les minuscules et les majuscules.

```
def chiffrer_vigenere_etendu(texte, cle):
    resultat = ""
    index_lettre = 0


    return resultat
```

Testez :

```
print(chiffrer_vigenere_etendu("z", "c"))
```

La lettre `c` correspond au décalage 2. Donc le résultat doit être :

```
B
```

Exercice 8 : Déchiffrement avec minuscules et majuscules

Écrivez maintenant la fonction inverse.

```
def dechiffrer_vigenere_etendu(texte_chiffre, cle):
    resultat = ""
    index_lettre = 0



    return resultat
```

Test complet :

```
message = "Bonjour zZ"
cle = "cA"

texte_chiffre = chiffrer_vigenere_etendu(message, cle)
texte_dechiffre = dechiffrer_vigenere_etendu(texte_chiffre, cle)

print("Message initial :", message)
print("Texte chiffre   :", texte_chiffre)
print("Texte dechiffre :", texte_dechiffre)
```

Travail attendu à la fin de la séance

À la fin de la séance, vous devez avoir au minimum :

- la fonction `lettre_vers_pos` ;
- la fonction `pos_vers_lettre` ;
- la fonction `chiffrer_vigenere` ;
- la fonction `dechiffrer_vigenere` ;
- un test montrant qu'un message chiffré peut être correctement déchiffré.